

Based on the technical training and Q&A transcript from the Zoho Catalyst hands-on deployment session, here are the critical architecture mandates, runtime configurations, implementation patterns, and hidden limitations required to align your **Problem Statement 1 (Intelligent Conversational AI)** prototype with the official evaluation parameters.

1. Native Zoho Catalyst Architecture & Setup

- **CLI Initialization Strategy:** Projects must be managed locally via the Catalyst CLI (zcatalyst-cli) via npm install -g zcatalyst-cli followed by catalyst init.
- **Component Composition:** The system configuration requires initializing two decoupling stacks:
 - **Backend Compute:** Serverless Functions configured as an **Advanced IO function** using the **NodeJS 24 runtime** pool.
 - **Web Client Hosting:** Deployed natively on **Catalyst Slate** (front-end static/SPA hosting) or connected directly to GitHub repositories via the console for automated build processing and CI/CD pipelines.
- **Cross-Origin Resource Sharing (CORS) Mandate:** Because Slate web client URLs and Serverless Function endpoints run on independent subdomains, you must explicitly whitelist the front-end production app URL inside the **Cloudscale -> Authentication -> Whitelisting** panel and enable CORS. Failing to do this causes catastrophic fetch failures at the client side.

2. Serverless Runtime & Execution Guardrails

- **The 30-Second Execution Wall:** Advanced IO Serverless Functions and AppSail instances have a hard, unalterable **30-second timeout limit**. Complex multi-node LLM pipelines or deep semantic generation workflows *will* face gateway timeouts.
- **The Job-Scheduling Workaround:** To bypass the 30-second execution limit for heavy background compute tasks (e.g., recursive data synthesis, batch relationship mining), you must utilize **Catalyst Job Functions**.
 - Job Functions provide an extended **15-minute timeout window**.
 - *Implementation Pattern:* Implement a dynamic Cron SDK inside your main API function to programmatically trigger a background Catalyst Job Function pool, tracking state asynchronously.
- **Independent Dependency Tree:** Each Serverless Function directory contains its isolated package.json manifest. Shared packages must be hard-coded and installed inside each function's respective folder (functions/<function_name>/) explicitly before deployment using catalyst deploy.

3. Storage & Access Control Mapping

- **Data Store Configuration:** The relational FIR system must map cleanly into the Catalyst Data Store. All baseline strings (FIR number, crime type, district details, timestamp vectors, status, accused/victim arrays, and descriptive logs) can utilize the text data type.
- **Explicit Role Scopes (Crucial Rule):** By default, initialized tables strip application users of write privileges (no privileges to perform this action). During seeding or administrative testing phases, you must manually navigate to **Cloudscale -> Data Store -> Scopes and**

Permissions to grant Insert and Update access, then safely restrict client-side privileges to Select in production to prevent data injection.

4. Generative AI Engine & QuickML Workflows

- **Foundational LLM Layer:** The architecture utilizes the in-house **GLM 4.7 Flash** model for structured reasoning and data transformations, and the **Qwen 3.6 (Vision)** model for multimodal image processing.
- **RAG Engine Integration:** Document grounding is isolated via the **QuickML Retrieval-Augmented Generation (RAG)** feature. PDF documents containing structural data are assigned a unique alphanumeric document ID upon console upload, which must be passed explicitly into environment variables (`quickl_rag_documents`).
- **Endpoint Access & Scope Authorization:** QuickML API requests must be explicitly signed using programmatic OAuth connections. You are required to establish an internal connection profile under **Cloudscale -> Connections**, mapping the precise authorization scope: `quickml.deployment.read`
- **Environment Management:** Secure values like `PROJECT_ID`, `ORGANIZATION_ID`, and model hyper-parameters (temperature, `max_tokens`, and optional model reasoning chains) must be hard-coded into the `catalyst-config.json` configuration array nested inside the serverless root directory.

5. Multilingual Pipelines & Voice Chaining

- **Indic Language Core:** The QuickML pipeline natively supports Indic language transformations (Kannada, Hindi, Telugu, Tamil).
- **The Chained Multimodal Voice Blueprint:** To satisfy the problem statement's strict text-and-voice dual capability across English and Kannada, you must stitch together a multi-stage execution chain within QuickML:
 1. **Stage 1:** Ingest regional speech vectors using the **In-House Speech-to-Text Transcription** model.
 2. **Stage 2:** Route the raw localized text through the **Text Translation Engine** to ground context in English (highly optimized for knowledge base parsing).
 3. **Stage 3:** Execute the semantic query via GLM 4.7/RAG against the Data Store or target PDF.
 4. **Stage 4:** Pipe the resulting English generation back through the translation engine to match the original Indic language.
 5. **Stage 5:** Synthesize the output text token stream into a fluid regional voice response utilizing the **Text-to-Audio Synthesis** model.

6. Rate Limiting & Failures

- **Rate-Limit Ceiling:** When the platform's API threshold ceilings are hit, all inbound execution requests are programmatically blocked. The system drops incoming packets into a **10-minute lock-out period**, after which the rate limits automatically refresh. Your client-side code must implement robust exponential backoff handling to manage this operational window during grading.

Reference link: [📺 KSP Datathon 2026 | Hands-on Workshop with Catalyst](#)